

Graph Algorithm exam

Time: 1 hour 45 minutes

Problem 1. – 2p

Consider the following algorithm:

Algorithm name: Maximum Clique by heuristic

Input:

g: an undirected graph

Output:

max_clique: a set containing the vertices that form a maximum clique

Algorithm:

```
while g.get_v() > 0:
    vertices = g.get_vertices()
    max = vertices[0]
    max_neighbors = g.neighbors(max)
    for i from 1 to v-1:
        new_neighbors = g.neighbors(vertices[i])
        if len(new_neighbors) > len(max_neighbors):
            max = vertices[i]
            max_neighbors = set(new_neighbors)
        end_if
    end_for
    max_neighbors.add(max)
    is_clique = True
    for neighbor in max_neighbors:
        new_neighbors = set(g.neighbors(neighbor))
        new_neighbors.add(neighbor)
        if @new_neighbors does not contain all max_neighbors:
            is_clique = False
            break
        end_if
    end_for
    if is_clique:
        return max_neighbors
    end_if
    g.remove_vertex(max)
end_while
return empty Set
end_algorithm
```

Analyze the time complexity of the algorithm in the following cases:

- a) g is implemented using a list of neighbors
- b) g is implemented using an adjacency matrix

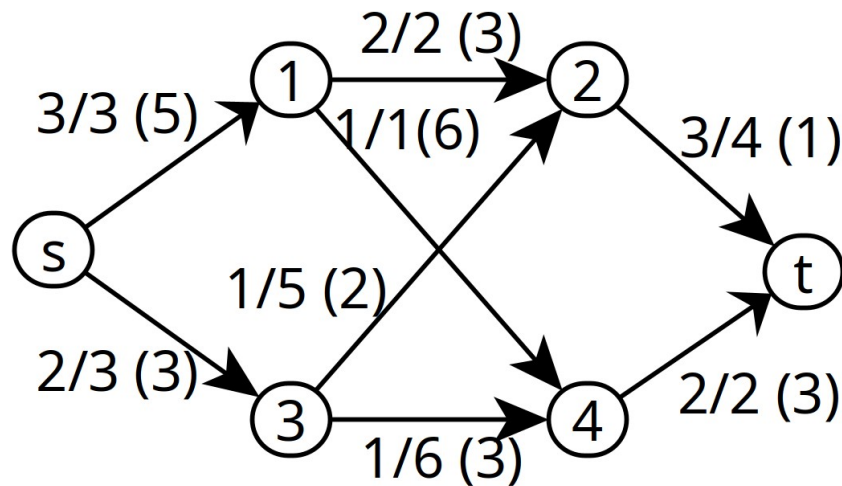
You have to properly justify the complexity of the algorithm simply writing that the algorithm is in some complexity will not be considered an answer.

Notes:

- you can consider that “get_vertices” is in $\Theta(v)$ in both cases.
- you can use “k” as a variable for the number of vertices in the clique that is found
- you can consider that checking if 1 set contains another is done in $\Theta(n_1 + n_2)$ where n_1 and n_2 are the sizes of the 2 sets.
- Creating a set from a list is $\Theta(n)$ where n is the size of the list.

Problem 2. – 3p

Consider the following flow network with weight and a flow f . On the edges there are three number $f / c (w)$, where f is the flow, c is the capacity and w is the cost (weight). For example, the edge (3, 4) has flow 1, capacity 5 and cost 2.



Exercises:

- 1) What is the total flow ($|f|$) of this network? (0.5 p)
- 2) What is the cut with minimum capacity? (0.5p)
- 3) Is the given flow maximal (is it the highest possible total flow for this network)? Why? (0.25 p)
- 4) Draw the residual network for this network flow and the flow f . (0.75p)
- 5) Explain how the residual network can be used to find a flow that has a lower cost. Use the method to find a different flow for the example. (1p)

Problem 3 – 4p

Create an algorithm that, given a number k and a connected, undirected graph, returns a list with all the clusters of size k that have the density greater than 0.8 (each cluster is an induced subgraph of the given graph).

Density of a graph is defined as e/v .

The time complexity of the algorithm does not matter.

Write the algorithm either in Python or pseudocode – same style of pseudocode as in the lecture notes.

Important: the input graph only has implemented the methods described below. You must use the input graph with those methods, you do not know how the graph is implemented (e.g. if it is an adjacency matrix or list of neighbors), you cannot access the internal structure of the graph and you cannot create new methods for the graph. When solving the problem you **must** use only these methods as described (it is not required to use all of them).

It is allowed to create auxiliary algorithms or functions. Also, you are allowed to use other data structures (list, stack, queue, set, map etc.) and you can consider that they are implemented efficiently.

Read these before starting on problem 3. The graph has the following methods:

- **get_vertices()** – returns a list with the vertices of the graph
- **get_v()** – returns the number of vertices (v)
- **get_e()** – returns the number of edges (e)
- **add_edge(v_1, v_2)** – adds the edge (v_1, v_2) to the graph
- **add_vertex(v)** – adds the vertex v to the graph
- **neighbors(v)** – returns a list of the neighbors of vertex v . The list contains the other vertices (e.g. if the edges are $[(1,2), (3,5), (3,2)]$ then `get_neighbors(3)` will return `[5,2]`)

Default – 1p